

Développement multiplateforme

Flutter

Laurent Bellon 2026-2027

1-1-1 Présentation de l'écosystème : Flutter vs React Native vs Natif

Pour choisir une stratégie de développement mobile, il est nécessaire de comprendre comment chaque approche interagit avec le système d'exploitation (OS) et le matériel.

1. Comparatif des approches techniques

Le choix entre ces technologies repose sur la manière dont le code est exécuté et rendu à l'écran.

Caractéristique	Natif (Swift/Kotlin)	React Native	Flutter
Langage	Swift / Kotlin	JavaScript / TypeScript	Dart
Rendu	Composants natifs de l'OS	Composants natifs via un "Bridge"	Moteur graphique propre (Skia/Impeller)
Performance	Maximale	Bonne (dépend du Bridge)	Excellente (compilé en code machine)
UI	Identique à l'OS	Proche du natif	Dessinée pixel par pixel

Fonctionnement détaillé

Natif

Le code accède directement aux API de l'OS. Il n'y a aucune couche d'abstraction. C'est l'approche la plus performante pour les applications nécessitant un accès intensif au matériel (Bluetooth, capteurs complexes, traitement vidéo).

React Native

Le code JavaScript communique avec les composants natifs via un "pont" (Bridge). Chaque interaction utilisateur doit traverser ce pont, ce qui peut créer des goulots d'étranglement lors d'animations complexes ou de calculs intensifs.

Flutter

Flutter utilise son propre moteur de rendu (Impeller). Il ne demande pas à l'OS de dessiner un bouton : il dessine lui-même le bouton sur un canevas. Cela garantit une cohérence visuelle totale sur toutes les versions d'Android et d'iOS.

2. Cas d'usage professionnels

Développement Natif

- **Exemple** : Application de montage vidéo haute performance ou jeu 3D complexe.
- **Pourquoi** : Accès direct aux API de bas niveau et gestion fine de la mémoire.

React Native

- **Exemple** : Application de e-commerce ou réseau social interne.
- **Pourquoi** : Réutilisation des compétences d'une équipe web (React) et écosystème de bibliothèques JavaScript mature.

Flutter

- **Exemple** : Application bancaire ou outil métier multiplateforme (Mobile, Web, Desktop).
- **Pourquoi** : Besoin d'une identité visuelle forte et identique sur toutes les plateformes, avec une performance proche du natif.

3. Points de vigilance et bonnes pratiques

Erreurs fréquentes à éviter

- **Vouloir tout partager** : Essayer de partager 100% du code entre mobile et web. Bien que Flutter le permette, les contraintes UX diffèrent (clavier physique vs tactile, navigation).
- **Ignorer les spécificités de l'OS** : Même avec Flutter, une application doit respecter les conventions de design (Material Design pour Android, Cupertino pour iOS).

Recommandations

- **Évaluation des besoins** : Si l'application nécessite des mises à jour fréquentes de l'UI sans mise à jour de l'App Store, React Native (via CodePush) offre un avantage.
- **Performance** : Pour des interfaces fluides à 60/120 FPS, Flutter est actuellement la solution multiplateforme la plus robuste grâce à sa compilation AOT (Ahead-of-Time).
- **Maintenance** : Dart est un langage fortement typé, ce qui réduit drastiquement le nombre de bugs à l'exécution par rapport à JavaScript.

4. Synthèse pour le choix technologique

1. **Performance brute** : Natif.
2. **Vitesse de développement & Écosystème Web** : React Native.
3. **Cohérence visuelle & Performance multiplateforme** : Flutter.

Sources

- [Flutter Documentation - Architectural Overview](#)
- [React Native Documentation - Architecture](#)

1-1-2 Architecture technique : Moteur de rendu et langage Dart

La performance de Flutter repose sur deux piliers : son langage de programmation, Dart, et son moteur de rendu graphique.

1. Le langage Dart : Moteur de la logique

Dart est un langage optimisé pour les interfaces utilisateur (UI). Il propose deux modes de compilation distincts selon l'environnement :

- **JIT (Just-In-Time)** : Utilisé lors du développement. Il permet le *Hot Reload*, injectant les modifications de code dans l'application en cours d'exécution sans perdre l'état de l'application.
- **AOT (Ahead-Of-Time)** : Utilisé pour la production. Le code est compilé en instructions machine natives (ARM ou x64), garantissant une exécution rapide et prévisible.

Caractéristiques clés

- **Typage fort** : Détecte les erreurs de type à la compilation.
- **Null Safety** : Le système de type garantit qu'une variable ne peut pas être `null` sauf si elle est explicitement déclarée comme telle, éliminant les erreurs de type `NullPointerException`.
- **Asynchronisme** : Basé sur le modèle `Future` et `async/await`, idéal pour les appels API sans bloquer l'interface.

2. Le moteur de rendu : De Skia à Impeller

Flutter ne délègue pas le rendu des composants aux widgets natifs de l'OS. Il utilise un moteur graphique pour dessiner chaque pixel.

- **Skia** : Moteur historique 2D open-source. Il transforme les instructions Flutter en commandes graphiques compréhensibles par le GPU.
- **Impeller** : Le nouveau moteur de rendu par défaut (sur iOS et progressivement Android). Il résout les problèmes de "jank" (saccades) liés à la compilation des shaders lors de l'exécution en pré-compilant ces derniers.



3. Comparatif des modes de compilation

Mode	Environnement	Avantage principal
JIT	Développement	Hot Reload (productivité)
AOT	Production	Performance native (vitesse)

4. Bonnes pratiques et points de vigilance

Utilisation de l'asynchronisme

Pour éviter de figer l'interface lors d'un appel réseau (ex: récupération de données depuis une API REST), utilisez systématiquement `async/await`.

```
// Exemple : Appel API asynchrone
Future<void> fetchUserData() async {
  try {
    final response = await http.get(Uri.parse('https://api.entreprise.com/user'));
    if (response.statusCode == 200) {
      // Traitement des données
    }
  } catch (e) {
    // Gestion des erreurs réseau
  }
}
```

Points de vigilance

- **Gestion de la mémoire** : Bien que Dart possède un Garbage Collector, évitez de créer des objets inutiles dans la méthode `build()` des widgets, car celle-ci est appelée fréquemment.
- **Null Safety** : Adoptez le typage strict. Évitez l'usage excessif de `!`, qui force le déballage d'une valeur potentiellement nulle et peut provoquer un crash.
- **Performance des shaders** : Avec l'adoption d'Impeller, la fluidité est accrue. Assurez-vous de cibler les versions récentes de Flutter pour bénéficier des optimisations liées à ce moteur.

Erreurs fréquentes

- **Bloquer le thread principal** : Effectuer des calculs lourds (ex: traitement d'image, parsing JSON massif) directement dans le thread UI. Utilisez `compute()` pour déléguer ces tâches à un *Isolate* (thread séparé).
- **Ignorer les avertissements du compilateur** : Les warnings de typage sont souvent des indicateurs de bugs potentiels en production.

Sources

- [Dart Language Documentation - Null Safety](#)
- [Flutter Documentation - Impeller Rendering Engine](#)
- [Flutter Documentation - Understanding the Flutter Engine](#)

1-1-3 Concepts fondamentaux : Widgets Stateless vs Stateful

Dans Flutter, tout est widget. L'interface utilisateur est une arborescence de widgets qui décrivent la configuration de l'UI. La distinction entre `StatelessWidget` et `StatefulWidget` est le fondement de la gestion de l'affichage.

1. StatelessWidget : L'affichage immuable

Un `StatelessWidget` est un widget qui ne change pas au cours de la durée de vie de l'application. Il ne dépend que des paramètres reçus lors de sa création (ses `properties`).

- **Fonctionnement** : Il est construit une seule fois. Si ses paramètres changent, le widget est détruit et reconstruit avec de nouvelles valeurs.
- **Cas d'usage** : Icônes, textes statiques, boutons de navigation simples, éléments de mise en page (padding, colonnes).

```
class SimpleLabel extends StatelessWidget {  
  final String text;  
  const SimpleLabel({super.key, required this.text});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(text);  
  }  
}
```

2. StatefulWidget : La gestion de l'état dynamique

Un `StatefulWidget` est utilisé lorsque l'interface doit réagir à des changements de données internes (ex: saisie utilisateur, réponse API, animation).

- **Fonctionnement** : Il se compose de deux classes :
 1. La classe `StatefulWidget` (immuable).
 2. La classe `State` (mutable), qui contient la logique et les données.
- **Cycle de vie** : Lorsque les données changent, l'appel à `setState()` notifie le framework que l'état a été modifié, déclenchant une reconstruction de la méthode `build()`.

```
class CounterWidget extends StatefulWidget {  
  const CounterWidget({super.key});  
  
  @override  
  State<CounterWidget> createState() => _CounterWidgetState();  
}  
  
class _CounterWidgetState extends State<CounterWidget> {  
  int _counter = 0;  
  
  void _increment() {  
    setState(() {  
      _counter++; // Mise à jour de l'état et déclenchement du rebuild  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(onPressed: _increment, child: Text('Compteur: $_counter'));  
  }  
}
```

3. Comparatif technique

Caractéristique	StatelessWidget	StatefulWidget
État interne	Aucun	Oui (via la classe <code>State</code>)
Performance	Optimale (léger)	Plus lourd (gestion du cycle de vie)
Réactivité	Non	Oui (via <code>setState</code>)

4. Bonnes pratiques et points de vigilance

Stratégie de découpage

- **Remonter l'état** : Ne placez pas `StatefulWidget` à la racine de votre application. Gardez les `StatefulWidget` le plus bas possible dans l'arborescence pour limiter la zone de reconstruction lors d'un `setState()`.
- **Séparation des responsabilités** : Un widget doit se concentrer sur l'affichage. La logique métier (appels API, calculs complexes) doit être déportée dans des classes dédiées (ex: services, contrôleurs).

Erreurs fréquentes à éviter

- `setState` **inutile** : Appeler `setState` pour modifier une variable qui n'est pas utilisée dans la méthode `build`.
- **Logique métier dans `build`** : Ne jamais effectuer d'appels réseau ou de calculs lourds directement dans la méthode `build()`. Elle est appelée très fréquemment par le framework.
- **Oublier `dispose()`** : Pour les ressources comme les `AnimationController` ou les `TextEditingController`, il est obligatoire d'appeler `dispose()` dans la méthode éponyme du `State` pour éviter les fuites de mémoire.

Sources

- [Flutter Documentation - Introduction to Widgets](#)
- [Flutter Documentation - StatefulWidget class](#)

1-2-1 Configuration du SDK Flutter et de l'éditeur

La mise en place d'un environnement de développement robuste est la première étape pour garantir une productivité optimale. Flutter nécessite le SDK (Software Development Kit) et un éditeur configuré avec les extensions appropriées.

1. Installation du SDK Flutter

Le SDK Flutter contient les outils nécessaires pour compiler, déboguer et déployer vos applications.

Étapes d'installation

1. **Téléchargement** : Récupérez la version stable la plus récente sur le site officiel flutter.dev.
2. **Extraction** : Décompressez l'archive dans un répertoire permanent (ex: `C:\src\flutter` sur Windows ou `~/development/flutter` sur macOS/Linux).
3. **Configuration du PATH** : Ajoutez le sous-dossier `bin` du répertoire Flutter à votre variable d'environnement `PATH`. Cela permet d'exécuter la commande `flutter` depuis n'importe quel terminal.
4. **Vérification** : Exécutez la commande `flutter doctor` dans votre terminal. Cet outil analyse votre environnement et liste les dépendances manquantes (ex: outils Android, Xcode, simulateurs).

2. Choix et configuration de l'éditeur

Deux éditeurs sont principalement utilisés dans le milieu professionnel :

Visual Studio Code (Recommandé pour la légèreté)

VS Code est l'éditeur le plus utilisé pour Flutter en raison de sa rapidité et de son écosystème d'extensions.

- **Extension indispensable** : Installez l'extension officielle "**Flutter**" (qui inclut automatiquement l'extension "**Dart**").
- **Avantages** : Démarrage rapide, intégration native du terminal, excellente gestion du *Hot Reload*.

Android Studio (Recommandé pour le débogage complexe)

Android Studio est un IDE complet basé sur IntelliJ.

- **Configuration** : Installez les plugins "Flutter" et "Dart" via le gestionnaire de plugins.
- **Avantages** : Outils de profilage mémoire et CPU très avancés, gestion intégrée des émulateurs Android, assistant de configuration pour les SDK natifs.

3. Comparatif des outils

Fonctionnalité	VS Code	Android Studio
Consommation RAM	Faible	Élevée
Vitesse d'ouverture	Très rapide	Lente
Outils de profilage	Basiques	Avancés
Configuration	Simple	Complète

4. Bonnes pratiques professionnelles

- **Utilisation de flutter doctor** : Avant chaque début de projet ou après une mise à jour du SDK, exécutez cette commande pour identifier les problèmes de configuration (ex: licences Android non acceptées).
- **Gestion des versions** : Utilisez un outil comme `fvm` (Flutter Version Management) pour gérer différentes versions de Flutter par projet. Cela évite les conflits lorsque vous travaillez sur plusieurs applications avec des contraintes de version différentes.
- **Emulateur vs Appareil physique** : Privilégiez le développement sur un appareil physique (mode développeur activé) pour tester les performances réelles et les interactions tactiles. Utilisez les émulateurs pour tester différentes tailles d'écran et versions d'OS.

5. Erreurs fréquentes à éviter

- **Installer Flutter dans un dossier système** : Évitez les dossiers comme `C:\Program Files` qui nécessitent des droits administrateur pour les mises à jour du SDK.
- **Ignorer les messages de `flutter doctor`** : Les erreurs concernant les licences Android (`flutter doctor --android-licenses`) sont fréquentes et bloquent la compilation.
- **Oublier de mettre à jour le SDK** : Flutter évolue rapidement. Utilisez `flutter upgrade` régulièrement pour bénéficier des dernières optimisations du moteur Impeller et des correctifs de sécurité.

Sources

- [Documentation officielle : Installation de Flutter](#)
- [Documentation officielle : Configuration de l'éditeur](#)
- [FVM \(Flutter Version Management\)](#)

1-2-2 Utilisation de 'flutter doctor' et gestion des émulateurs

La maîtrise de l'environnement de développement repose sur la capacité à diagnostiquer les problèmes de configuration et à tester l'application sur des cibles variées.

1. Diagnostic avec `flutter doctor`

La commande `flutter doctor` est l'outil de diagnostic principal. Elle vérifie l'installation du SDK, la présence des outils de build (Android SDK, Xcode, Chrome, etc.) et l'état des IDE.

Fonctionnement

- **Analyse** : Le framework interroge le système pour localiser les dépendances nécessaires.
- **Rapport** : Il affiche une liste de coches (✅) pour les éléments configurés et de croix (❌) pour les éléments manquants ou mal configurés.
- **Résolution** : Le rapport indique souvent la commande exacte à exécuter pour corriger une erreur (ex: acceptation des licences Android).

Exemple de flux de travail

```
# Lancer le diagnostic
```

```
flutter doctor
```

```
# Exemple de correction courante pour Android
```

```
flutter doctor --android-licenses
```

2. Gestion des émulateurs et simulateurs

Pour tester une application, vous devez disposer d'un environnement d'exécution.

- **Émulateur Android (AVD)** : Créé via Android Studio (Device Manager). Il permet de simuler différentes versions d'Android et différentes tailles d'écran (tablettes, téléphones pliables).
- **Simulateur iOS** : Disponible uniquement sur macOS via Xcode. Il simule les appareils Apple.
- **Appareils physiques** : Connectés via USB ou Wi-Fi (mode débogage activé).

Comparatif des environnements de test

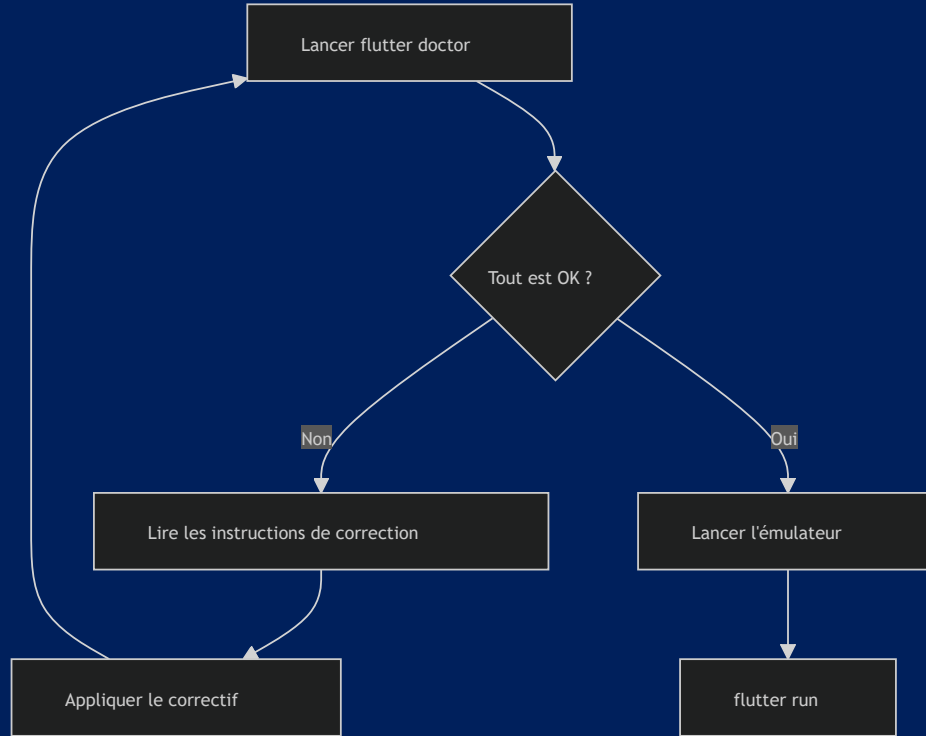
Type	Avantages	Inconvénients
Émulateur	Gratuit, multi-versions, facile à configurer	Consomme beaucoup de RAM, pas de capteurs réels
Appareil physique	Performance réelle, capteurs (GPS, Bluetooth)	Coûteux, gestion des câbles/batterie

3. Bonnes pratiques professionnelles

- **Automatisation** : Intégrez `flutter doctor` dans vos scripts de CI/CD pour vérifier que l'environnement de build est conforme avant de lancer les tests automatisés.
- **Nettoyage** : Si une application ne se compile plus après une mise à jour de dépendances, utilisez `flutter clean` suivi de `flutter pub get` avant de chercher une erreur de configuration complexe.
- **Diversité des tests** : Configurez au moins deux émulateurs : un avec une version d'OS récente et un avec une version minimale supportée par votre projet.

4. Points de vigilance et erreurs fréquentes

- **Licences Android** : L'erreur la plus fréquente après une installation est l'absence d'acceptation des licences. `flutter doctor` le détecte systématiquement.
- **Virtualisation (VT-x/AMD-V)** : Si les émulateurs Android sont extrêmement lents, vérifiez que la virtualisation est activée dans le BIOS/UEFI de votre machine.
- **Espace disque** : Les images d'émulateurs (AVD) occupent plusieurs gigaoctets. Nettoyez régulièrement les images inutilisées dans le Device Manager d'Android Studio.
- **Conflits de ports** : Si vous lancez plusieurs émulateurs, assurez-vous que votre machine dispose de suffisamment de ressources CPU, sous peine de voir le *Hot Reload* ralentir considérablement.



Sources

- [Flutter Documentation - Testing and debugging](#)
- [Flutter Documentation - Android setup](#)

2-1-1 Widgets de structure : Row, Column, Stack, Container

La composition d'interfaces dans Flutter repose sur l'imbrication de widgets. Les widgets de structure permettent d'organiser, d'aligner et de styliser les éléments visuels.

1. Organisation linéaire : Row et Column

Ces widgets permettent de disposer des éléments de manière unidimensionnelle.

- **Row (Ligne)** : Aligne les enfants horizontalement.
- **Column (Colonne)** : Aligne les enfants verticalement.

Propriétés clés

- `mainAxisAlignment` : Définit l'alignement sur l'axe principal (horizontal pour Row, vertical pour Column).
- `crossAxisAlignment` : Définit l'alignement sur l'axe secondaire (vertical pour Row, horizontal pour Column).

```
// Exemple : Une ligne avec deux éléments espacés
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Icon(Icons.person),
    Text("Profil utilisateur"),
  ],
)
```

2. Superposition : Stack

Le widget `Stack` permet de placer des widgets les uns au-dessus des autres. Il est idéal pour superposer du texte sur une image ou afficher un badge sur une icône.

- **Positionnement** : Utilisez le widget `Positioned` comme enfant direct d'une `Stack` pour définir précisément les coordonnées (top, bottom, left, right).

```
Stack(  
  children: [  
    Container(  
      width: 300,  
      height: 200,  
      color: Colors.grey,  
    ),  
  ],  
)
```

--> Suite -->

```
Positioned(
  left: 20,
  top: 30,
  child: Text(
    'Bonjour Flutter',
    style: TextStyle(fontSize: 20),
  ),
),

Positioned(
  right: 10,
  bottom: 10,
  child: ElevatedButton(
    onPressed: () {},
    child: Text('Valider'),
  ),
),
],
)
```

Ici, le texte est donc placé à (20, 30) dans le Stack, tandis que le bouton est positionné à 10 px du bas et 10 px de la droite.

3. Le couteau suisse : Container

Le `Container` est un widget polyvalent qui combine plusieurs fonctionnalités : décoration, padding, marges, taille et alignement.

- **Usage** : À utiliser pour ajouter un fond coloré, des bordures arrondies ou des espacements autour d'un widget.
- **Attention** : Un `Container` sans enfant tente de prendre toute la taille disponible, alors qu'un `Container` avec un enfant s'adapte à la taille de celui-ci.

4. Comparatif des widgets de structure

Widget	Rôle principal	Axe de disposition
Row	Alignement horizontal	Horizontal
Column	Alignement vertical	Vertical
Stack	Superposition	Z-index (profondeur)
Container	Décoration et mise en forme	N/A (boîte unique)

5. Bonnes pratiques et points de vigilance

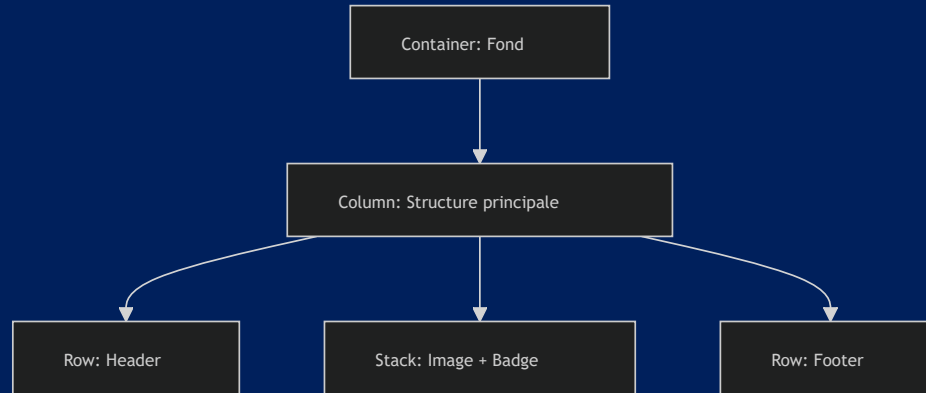
Erreurs fréquentes

- **Débordement (Overflow)** : Placer trop d'éléments dans une `Row` ou `Column` sans utiliser `Expanded` ou `Flexible` provoque une erreur de type "RenderFlex overflowed".
- **Imbrication excessive** : Créer des arbres de widgets trop profonds rend le code difficile à maintenir. Extrayez les sous-parties de votre UI dans des méthodes ou des classes de widgets dédiées.
- **Utilisation abusive de Container** : Si vous n'avez besoin que d'un padding, utilisez le widget `Padding` plutôt qu'un `Container`. Il est plus léger.

Points de vigilance

- **Flexibilité** : Utilisez `Expanded` pour forcer un enfant à occuper tout l'espace restant dans une `Row` ou `Column`.
- **Performance** : Le `Stack` est plus coûteux en ressources que `Row` ou `Column`. Utilisez-le uniquement lorsque la superposition est nécessaire.

Exemple de structure complexe



6. Bonnes pratiques

- **Design System** : Pour des interfaces cohérentes, créez vos propres widgets de structure réutilisables (ex: `AppButton`, `ProfileCard`) plutôt que de répéter des `Container` avec les mêmes propriétés de décoration.
- **Lisibilité** : Utilisez des commentaires pour délimiter les blocs de widgets complexes dans vos fichiers Dart.

Sources

- [Flutter Documentation - Layout widgets](#)
- [Flutter Documentation - Box model](#)

2-1-2 Gestion de l'espace : Padding, Margin, SizedBox, Expanded

La maîtrise de l'espacement est déterminante pour la qualité visuelle d'une interface. Flutter propose des outils spécifiques pour contrôler les dimensions et les zones vides.

1. Padding et Margin

Dans Flutter, le concept de "margin" n'existe pas en tant que propriété native de tous les widgets.

- **Padding** : Ajoute de l'espace à l'intérieur des limites d'un widget. Le widget `Padding` est la méthode standard pour appliquer cet espacement.
- **Margin** : Pour simuler une marge (espace extérieur), on utilise généralement un widget `Padding` parent ou les propriétés `margin` du widget `Container`.

```
Container(  
  margin: const EdgeInsets.all(16.0),  
  child: const Text('Contenu avec marge externe'),  
)
```

- **Container** : Widget conteneur modulable permettant la mise en forme et le positionnement.
- **Margin** : `const EdgeInsets.all(16.0)` : Crée un espace extérieur de 16 pixels tout autour du conteneur.
- **child** : `const Text(...)` : Le widget enfant placé à l'intérieur du conteneur.

```
Padding(  
  padding: const EdgeInsets.symmetric(horizontal: 12.0, vertical: 8.0),  
  child: const Text('Contenu avec espace interne'),  
)
```

- **Padding** : Widget dédié exclusivement à l'ajout d'espace interne autour de son enfant.
- **EdgeInsets.symmetric(...)** : Applique une marge interne distincte : 12 pixels sur les côtés (horizontal) et 8 pixels en haut/bas (vertical).

2. Expanded : Gestion dynamique

Le widget `Expanded` est indispensable pour occuper l'espace disponible au sein d'une `Row` ou d'une `Column`.

- **Fonctionnement** : Il force son enfant à s'étendre pour remplir l'espace restant sur l'axe principal.
- **Flex** : La propriété `flex` permet de définir des ratios (ex: un élément occupant 2/3 de l'espace et un autre 1/3).

```
Row(  
  children: [  
    Expanded(flex: 2, child: Container(color: Colors.blue)),  
    Expanded(flex: 1, child: Container(color: Colors.red)),  
  ],  
)
```

- **Row** : Aligner ses widgets enfants horizontalement sur une ligne.
- **Expanded** : Force l'enfant à occuper l'espace disponible dans la ligne.
- **flex: 2 / flex: 1** : Divise l'espace selon un ratio de 2 : 1 (66% de la largeur pour le premier conteneur, 33% pour le second).

3. Flex + SizedBox : Contrôle dimensionnel

Le `SizedBox` est un widget simple permettant de forcer des dimensions précises ou d'ajouter un espace vide entre deux éléments.

- **Espaceur** : Utilisé dans une `Row` ou `Column` pour créer un écart fixe.
- **Contrainte** : Utilisé pour forcer un enfant à avoir une largeur ou une hauteur spécifique.

```
Flex(  
  direction: Axis.horizontal,  
  children: [  
    Text('Nom'),  
    SizedBox(width: 20),  
    Text('Prénom'),  
  ],  
)
```

- **Flex** permet d'organiser des widgets horizontalement ou verticalement.
- **Axis.horizontal** → comme un `Row`
- **Axis.vertical** → comme un `Column`

4. Boutons cliquables

```
ElevatedButton(  
  onPressed: () => print('Action effectuée'),  
  child: const Text('Cliquez ici'),  
)
```

- **ElevatedButton** : Bouton matériel standard avec une légère ombre d'élévation.
- **onPressed: () => ...** : Fonction callback exécutée automatiquement lorsque l'utilisateur appuie sur le bouton.
- **child: const Text(...)** : Composant visuel affiché à l'intérieur du bouton (texte, icône, etc.).

5. Header / Footer

```
Scaffold(  
  appBar: AppBar(  
    title: Text('Mon application'),  
  ),  
  body: Center(  
    child: Text('Bienvenue'),  
  ),  
)
```

- **AppBar** représente généralement le header de l'application : titre, bouton retour, actions, etc.
- **Scaffold** fournit la structure générale de l'écran.

```
Scaffold(  
  body: Center(  
    child: Text('Accueil'),  
  ),  
  bottomNavigationBar: BottomNavigationBar(  
    items: const [  
      BottomNavigationBarItem(  
        icon: Icon(Icons.home),  
        label: 'Accueil',  
      ),  
      BottomNavigationBarItem(  
        icon: Icon(Icons.person),  
        label: 'Profil',  
      ),  
    ],  
  ),  
)
```

- **bottomNavigationBar** permet de placer une barre de navigation en bas de l'écran, souvent utilisée comme footer pour naviguer entre les principales sections.

6. Résumé

Élément	Rôle
<code>Padding</code>	Espace intérieur
<code>margin</code>	Espace extérieur
<code>Flex</code>	Organisation horizontale/verticale
<code>ElevatedButton</code>	Bouton interactif
<code>AppBar</code>	Header
<code>bottomNavigationBar</code>	Footer / navigation basse

7. Bonnes pratiques et points de vigilance

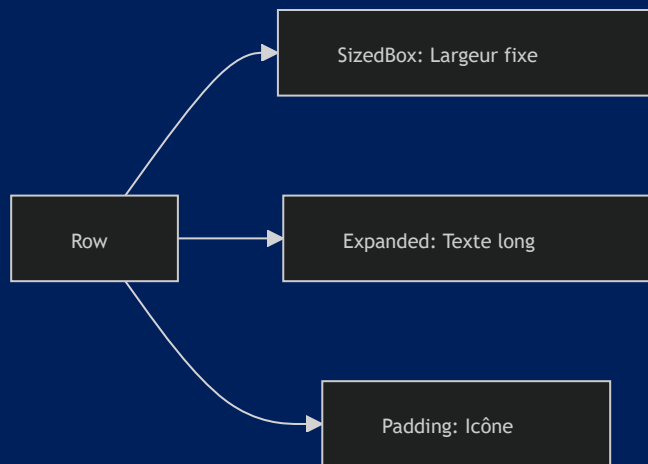
Erreurs fréquentes

- **SizedBox vs Padding** : Utiliser un `Container` avec un `padding` juste pour créer un espace vide est trop lourd. Préférez `SizedBox` pour des espaces simples ou `Padding` pour envelopper un widget existant.
- **Expanded en dehors d'un Flex** : Utiliser `Expanded` en dehors d'une `Row`, `Column` ou `Flex` provoquera une erreur à l'exécution.
- **Valeurs "en dur"** : Évitez de multiplier les valeurs numériques fixes (ex: `SizedBox(height: 20)`). Utilisez des constantes ou des variables de thème pour garantir la cohérence visuelle de l'application.

Points de vigilance

- **Flexibilité** : Si vous avez plusieurs `Expanded` dans une `Row`, ils se partageront l'espace disponible en fonction de leur valeur `flex`.
- **Débordement** : Si le contenu d'un `Expanded` est trop grand, il peut provoquer un débordement. Combinez-le avec des widgets de défilement (`SingleChildScrollView`) si nécessaire.

Exemple de mise en page professionnelle



8. Recommandations

- **Utilisation de `Spacer`** : Pour créer un espace flexible entre deux éléments dans une `Row` ou `Column`, le widget `Spacer` est une alternative plus lisible à `Expanded(child: SizedBox())`.
- **Constantes** : Définissez des espacements standards dans un fichier `constants.dart` (ex: `static const double spacing = 16.0;`) pour uniformiser le design de l'application.

Sources

- [Flutter Documentation - Padding class](#)
- [Flutter Documentation - Expanded class](#)
- [Flutter Documentation - SizedBox class](#)

3-1-1 Navigation impérative (Navigator 1.0) et routes nommées

La navigation dans Flutter repose sur le concept de pile (Stack). Le `Navigator` gère cette pile en ajoutant ou en supprimant des écrans.

1. Concepts fondamentaux : La pile de navigation

Le `Navigator` fonctionne comme une pile LIFO (*Last In, First Out*).

- **Push** : Ajoute un nouvel écran au sommet de la pile.
- **Pop** : Retire l'écran actuel pour revenir au précédent.

2. Navigation impérative (Navigator.push)

La méthode la plus directe consiste à utiliser `MaterialPageRoute`. Elle permet de définir dynamiquement l'écran de destination.

```
// Navigation vers un nouvel écran
Navigator.of(context).push(
  MaterialPageRoute(builder: (context) => const DetailsScreen()),
);

// Retour à l'écran précédent
Navigator.of(context).pop();
```

3. Routes nommées

Pour éviter de répéter les constructeurs de pages, on utilise une table de routage définie dans le `MaterialApp`.

Configuration

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const HomeScreen(),  
    '/details': (context) => const DetailsScreen(),  
  },  
);
```

Utilisation

```
// Navigation via le nom de la route  
Navigator.pushNamed(context, '/details');
```

4. Comparatif des approches

Approche	Avantages	Inconvénients
Push direct	Simple, typé, passage de paramètres facile	Couplage fort entre les écrans
Routes nommées	Centralisation, lisibilité	Passage de paramètres complexe

5. Gestion des paramètres

Avec les routes nommées, le passage de données complexes (objets métiers) est fastidieux. Utilisez

`ModalRoute.of(context)!.settings.arguments` pour récupérer les données transmises lors du `pushNamed`.

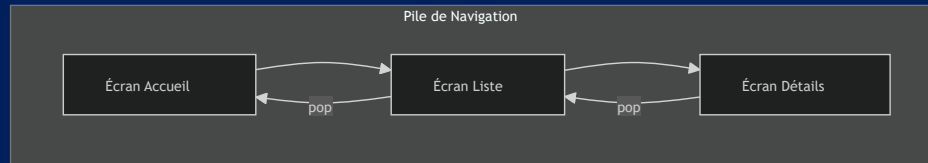
Erreurs fréquentes

- **Contexte invalide** : Appeler `Navigator` avec un `context` qui n'est pas un descendant du `Navigator` (souvent lors de l'utilisation d'un `Builder` ou d'un `async` trop long).
- **Oubli du `pop`** : Ne pas gérer le retour arrière, ce qui bloque l'utilisateur sur une page.

Points de vigilance

- **Profondeur de pile** : Une pile trop profonde consomme de la mémoire. Utilisez `pushReplacementNamed` pour remplacer l'écran actuel (ex: écran de login vers écran d'accueil) afin d'éviter que l'utilisateur ne puisse revenir en arrière sur l'écran de connexion.
- **Découplage** : Pour des applications de grande taille, la navigation impérative devient difficile à maintenir. Bien que le `Navigator 1.0` soit suffisant pour des besoins simples, envisagez des solutions basées sur des packages de routage (comme `go_router`) pour gérer les liens profonds (*deep linking*) et les états complexes.

6. Diagramme de la pile (Stack)



Sources

- [Flutter Documentation - Navigation and routing](#)
- [Flutter API - Navigator class](#)

3-1-2 Passage d'arguments lors de la transition

Le passage de données entre écrans est une nécessité pour construire des interfaces dynamiques, comme l'affichage des détails d'un produit sélectionné dans une liste.

1. Passage d'arguments via le constructeur (Navigation impérative)

C'est la méthode la plus directe et la plus sûre en termes de typage. Vous passez les données directement au constructeur de la page de destination.

```
// Écran source
Navigator.of(context).push(
  MaterialPageRoute(
    builder: (context) => DetailsScreen(productId: "123"),
  ),
);

// Écran destination
class DetailsScreen extends StatelessWidget {
  final String productId;
  const DetailsScreen({super.key, required this.productId});
  // ...
}
```

2. Passage d'arguments via les routes nommées

Lorsque vous utilisez `pushNamed`, vous devez transmettre les données via le paramètre `arguments`.

Envoi des données

```
Navigator.pushNamed(  
  context,  
  '/details',  
  arguments: {'id': '123', 'name': 'Produit A'},  
);
```

Récupération des données

Dans l'écran de destination, vous accédez aux arguments via `ModalRoute`.

```
final args = ModalRoute.of(context)!.settings.arguments as Map<String, String>;  
final productId = args['id'];
```

3. Comparatif des méthodes

Méthode	Typage	Complexité	Usage recommandé
Constructeur	Fort (Compile-time)	Faible	Navigation simple et directe
Routes nommées	Faible (Runtime)	Moyenne	Navigation centralisée, deep linking

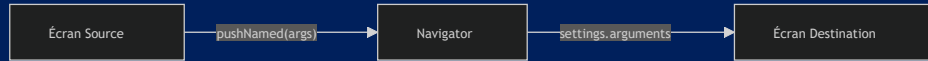
4. Bonnes pratiques

- **Utilisation de classes de données** : Au lieu de passer des `Map` ou des `dynamic`, créez des classes dédiées pour vos arguments. Cela permet de bénéficier de l'autocomplétion et de la sécurité du typage.
- **Validation** : Vérifiez toujours que les arguments reçus ne sont pas nuls avant de les utiliser, surtout avec `ModalRoute`.
- **Séparation des responsabilités** : L'écran de destination ne doit pas savoir comment les données sont récupérées. Il doit simplement recevoir les paramètres nécessaires via son constructeur.

5. Erreurs fréquentes et points de vigilance

- **Casting erroné** : Utiliser `as` sur un type incorrect lors de la récupération des arguments provoque un crash à l'exécution.
- **Arguments manquants** : Si une route nommée est appelée sans arguments alors que l'écran en attend, l'application échouera.
- **Données complexes** : Évitez de passer des objets métier très complexes ou des instances de services via la navigation. Préférez passer un identifiant (ID) et récupérer l'objet complet depuis un gestionnaire d'état ou une base de données locale.

6. Flux de transmission



Sources

- [Flutter Documentation - Passing arguments to a named route](#)
- [Flutter API - ModalRoute class](#)

3-1-3 Organisation du fichier de routes principal

La centralisation de la navigation est une étape déterminante pour la maintenabilité d'une application Flutter. Un fichier de routes bien structuré évite la dispersion de la logique de navigation dans l'ensemble du code source.

1. Centralisation des routes

Au lieu de définir les routes directement dans le `MaterialApp`, il est recommandé de les isoler dans une classe dédiée ou un fichier spécifique (ex: `app_router.dart`).

Structure recommandée

```
class AppRoutes {  
  static const String home = '/';  
  static const String details = '/details';  
  static const String settings = '/settings';  
  
  static Map<String, WidgetBuilder> get routes => {  
    home: (context) => const HomeScreen(),  
    details: (context) => const DetailsScreen(),  
    settings: (context) => const SettingsScreen(),  
  };  
}
```

2. Utilisation dans le MaterialApp

Cette approche permet de garder le fichier `main.dart` propre et lisible.

```
MaterialApp(  
  initialRoute: AppRoutes.home,  
  routes: AppRoutes.routes,  
);
```

3. Avantages de la centralisation

- **Maintenance facilitée** : Modifier le nom d'une route ou changer un écran se fait en un seul endroit.
- **Lisibilité** : Le `main.dart` ne contient plus la configuration technique des routes.
- **Réutilisation** : Les constantes de routes (`AppRoutes.home`) peuvent être utilisées partout dans l'application pour éviter les fautes de frappe lors des appels `pushNamed` .

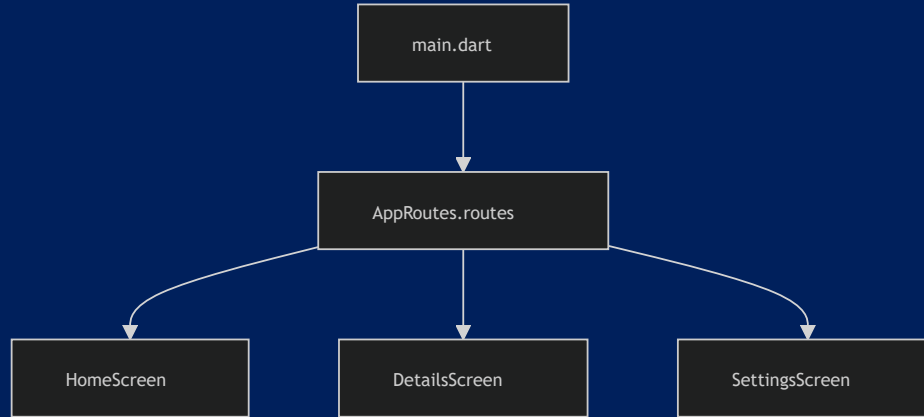
4. Bonnes pratiques

- **Utilisation de constantes** : Ne jamais utiliser de chaînes de caractères "en dur" (hardcoded) pour les noms de routes. Utilisez toujours les constantes définies dans votre classe `AppRoutes`.
- **Découpage par module** : Pour les applications de grande envergure, créez des fichiers de routes par fonctionnalité (ex: `auth_routes.dart`, `profile_routes.dart`) et fusionnez-les dans un fichier principal.
- **Gestion des routes inconnues** : Utilisez la propriété `onGenerateRoute` dans le `MaterialApp` pour gérer les cas où une route n'est pas trouvée, permettant d'afficher une page d'erreur personnalisée.

5. Erreurs fréquentes et points de vigilance

- **Importations circulaires** : Veillez à ce que votre fichier de routes n'importe pas des widgets qui, eux-mêmes, importent le fichier de routes.
- **Oubli de mise à jour** : Ajouter un écran sans l'enregistrer dans la table de routage est une source fréquente de bugs.
- **Complexité excessive** : Si votre fichier de routes dépasse quelques centaines de lignes, il est temps de le diviser par domaine fonctionnel.

6. Organisation logique



7. Recommandations

Bien que la méthode `routes` soit native et simple, elle montre ses limites pour des besoins avancés (protection de routes, redirection conditionnelle, paramètres complexes). Pour des projets professionnels, l'adoption d'un package comme `go_router` est devenue la norme. Il permet une configuration déclarative des routes, une meilleure gestion des états de navigation et une compatibilité native avec le Web (URL propres).

Sources

- [Flutter Documentation - Navigation and routing](#)
- [Go Router Package - Documentation](#)

4-1-1 Limites de setState pour les applications complexes

Le widget `StatefulWidget` et sa méthode `setState()` constituent le mécanisme de base pour mettre à jour l'interface utilisateur dans Flutter. Cependant, leur utilisation intensive dans des applications de grande envergure pose des problèmes structurels.

1. Fonctionnement de setState

`setState()` informe le framework que l'état interne d'un widget a changé, déclenchant ainsi une reconstruction de la méthode `build()` de ce widget et de ses enfants.

```
// Exemple simple : Compteur
setState(() {
  _counter++;
});
```

2. Limites dans les applications complexes

Couplage fort (Spaghetti Code)

Lorsque la logique métier est mélangée à la logique d'affichage dans le widget, le code devient difficile à tester et à maintenir.

"Prop Drilling" (Passage de paramètres en cascade)

Pour partager une donnée entre un widget parent et un petit-enfant situé plusieurs niveaux plus bas, vous devez passer cette donnée à travers chaque constructeur intermédiaire, même si ces widgets intermédiaires n'utilisent pas la donnée.

Reconstructions inutiles

`setState()` reconstruit tout le sous-arbre du widget. Dans une application complexe, cela peut entraîner des problèmes de performance si des widgets lourds sont reconstruits inutilement.

3. Comparatif : setState vs Gestion d'état externe

Caractéristique	setState	Gestion d'état (Provider/Bloc)
Portée	Locale au widget	Globale ou partagée
Logique métier	Mélangée à l'UI	Séparée de l'UI
Maintenance	Difficile à grande échelle	Facilitée par la séparation
Testabilité	Complexe (nécessite l'UI)	Simple (logique isolée)

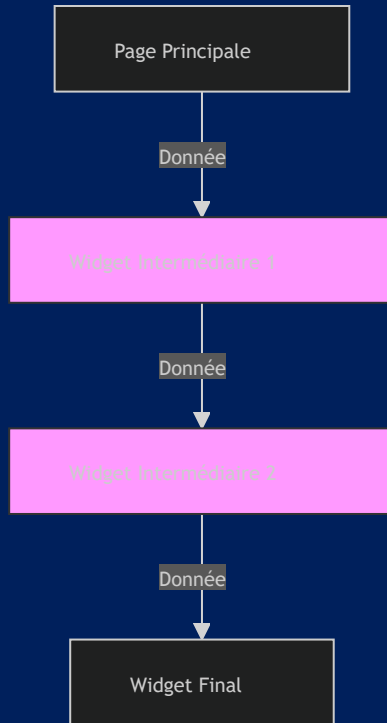
4. Cas d'usage : Quand éviter setState ?

- **Données partagées** : Lorsque plusieurs écrans ont besoin d'accéder à la même information (ex: profil utilisateur, panier d'achat).
- **Logique métier complexe** : Lorsque le traitement des données nécessite des appels API, des calculs ou des interactions avec des bases de données locales.
- **Applications multi-écrans** : Lorsque l'état doit persister lors de la navigation entre différentes routes.

5. Erreurs fréquentes et points de vigilance

- **Appel de `setState` après suppression** : Appeler `setState()` sur un widget qui a été retiré de l'arbre (ex: après une opération asynchrone) provoque une erreur. Vérifiez toujours `if (mounted)` avant.
- **Logique métier dans `build()`** : Effectuer des calculs lourds ou des appels réseau directement dans `build()` ou `setState()` ralentit l'interface.
- **Oubli de nettoyage** : Ne pas annuler les abonnements (streams, timers) dans la méthode `dispose()` crée des fuites de mémoire.

6. Illustration du problème de "Prop Drilling"



Les widgets en rose sont forcés de recevoir la donnée uniquement pour la transmettre au suivant.

7. Recommandations

- **Utilisez `setState` uniquement pour l'état éphémère** : C'est-à-dire l'état qui ne concerne qu'un seul widget (ex: état d'un champ de texte, animation locale, ouverture d'un menu déroulant).
- **Externalisez la logique métier** : Dès qu'une donnée doit être partagée ou modifiée par plusieurs composants, déplacez-la vers une classe de gestion d'état (comme `ChangeNotifier` avec `Provider`).
- **Priorisez la séparation des préoccupations** : Votre code doit être structuré de manière à ce que la logique métier puisse être testée sans instancier de widgets.

Sources

- [Flutter Documentation - Introduction to state management](#)
- [Flutter Documentation - Ephemeral vs App State](#)

4-1-2 Introduction à Provider : ChangeNotifier, ChangeNotifierProvider, Consumer

Le package `provider` est une solution recommandée par l'équipe Flutter pour la gestion d'état. Il repose sur le mécanisme d'héritage des widgets (`InheritedWidget`) tout en simplifiant considérablement son utilisation.

1. Les trois piliers de Provider

ChangeNotifier

C'est une classe simple qui étend `ChangeNotifier`. Elle contient les données de votre application et les méthodes pour les modifier. Lorsqu'une donnée change, vous appelez `notifyListeners()` pour avertir les widgets abonnés.

```
class CartModel extends ChangeNotifier {
  int _items = 0;
  int get items => _items;

  void add() {
    _items++;
    notifyListeners(); // Notifie les widgets abonnés
  }
}
```

ChangeNotifierProvider

C'est le widget qui injecte votre instance de `ChangeNotifier` dans l'arbre des widgets. Il doit être placé au-dessus des widgets qui ont besoin d'accéder aux données.

```
ChangeNotifierProvider(  
  create: (context) => CartModel(),  
  child: MyApp(),  
);
```

Consumer

C'est un widget qui écoute les changements du `ChangeNotifier`. Il reconstruit uniquement sa propre partie de l'interface lorsque `notifyListeners()` est appelé.

```
Consumer<CartModel>(  
  builder: (context, cart, child) {  
    return Text("Articles : ${cart.items}");  
  },  
);
```

2. Fonctionnement détaillé

1. **Injection** : `ChangeNotifierProvider` crée l'instance du modèle et la rend disponible dans le contexte.
2. **Abonnement** : Le `Consumer` cherche dans le contexte le `ChangeNotifier` correspondant.
3. **Notification** : Dès que `notifyListeners()` est appelé, le `Consumer` exécute à nouveau sa fonction `builder`.

3. Comparatif : Accès aux données

Méthode	Usage	Performance
<code>Consumer</code>	Recommandé pour reconstruire une partie précise	Optimale
<code>Provider.of<T>(context)</code>	Accès direct (sans écoute)	N/A
<code>Provider.of<T>(context, listen: true)</code>	Équivalent au <code>Consumer</code>	Optimale

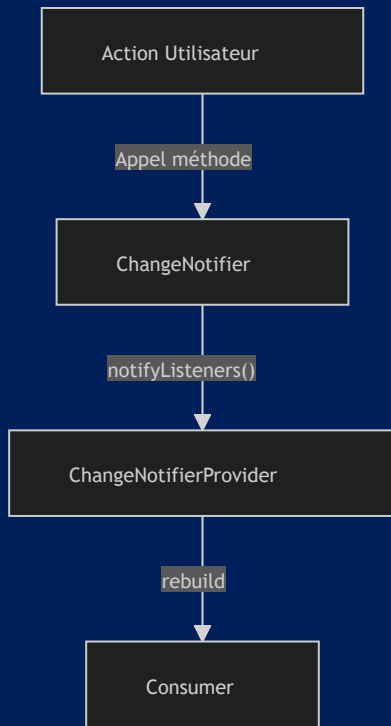
4. Bonnes pratiques

- **Placer le Provider au plus haut** : Si plusieurs écrans ont besoin de la donnée, placez le `ChangeNotifierProvider` au-dessus de `MaterialApp`.
- **Utiliser `Consumer` pour limiter les reconstructions** : Ne placez pas le `Consumer` à la racine de votre écran si seule une petite partie (ex: un compteur dans une barre d'outils) doit être mise à jour.
- **Séparation des modèles** : Créez un `ChangeNotifier` par domaine fonctionnel (ex: `AuthModel`, `ProductModel`, `CartModel`) plutôt qu'un seul modèle géant.

5. Erreurs fréquentes et points de vigilance

- **Oubli de `notifyListeners()`** : L'interface ne se mettra pas à jour si vous modifiez la donnée sans appeler cette méthode.
- **Provider introuvable** : Si vous essayez d'accéder à un `Provider` depuis un widget situé au-dessus de lui dans l'arbre, une erreur sera levée.
- **Reconstructions inutiles** : Utiliser `Provider.of<T>(context)` dans une méthode `build` complète déclenchera une reconstruction de tout le widget à chaque changement. Préférez le `Consumer` pour isoler la reconstruction.

6. Flux de données



7. Recommandation

Sources

- [Pub.dev - Provider package](#)
- [Flutter Documentation - Simple app state management](#)

4-1-3 Injection de dépendances simple

L'injection de dépendances (DI) est une technique permettant de fournir à un objet les services ou données dont il a besoin, plutôt que de les créer directement à l'intérieur de celui-ci. Avec `provider`, cette pratique devient native et très accessible.

1. Concept fondamental

Au lieu d'instancier un service (ex: `ApiService`) directement dans un widget, vous l'injectez via le `Provider`. Cela permet de découpler vos composants et de faciliter les tests unitaires en remplaçant facilement une implémentation réelle par une version "mock" (simulée).

2. Mise en œuvre avec Provider

Injection d'un service

Utilisez `Provider` (sans `ChangeNotifier`) pour injecter des objets qui ne nécessitent pas de notification de changement, comme des services API ou des dépôts de données.

```
// Injection au sommet de l'arbre
Provider<ApiService>(
  create: (context) => ApiService(),
  child: MyApp(),
);
```

Accès au service

Dans n'importe quel widget enfant, vous récupérez l'instance sans avoir à la passer via les constructeurs.

```
final apiService = Provider.of<ApiService>(context, listen: false);
apiService.fetchData();
```

3. Comparatif : Instanciation vs Injection

Approche	Dépendance	Testabilité
Instanciation directe	Forte (Couplage)	Difficile
Injection de dépendances	Faible (Découplage)	Facile (Mocking)

4. Cas d'usage

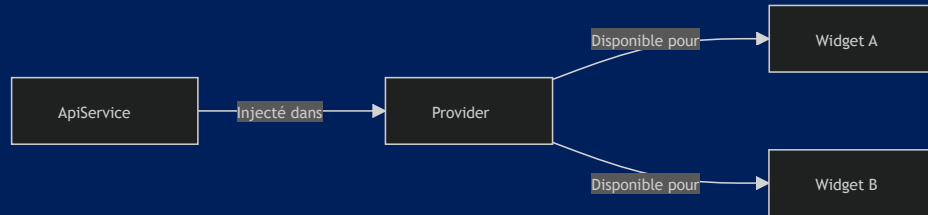
- **Services API** : Injecter un client HTTP configuré (ex: `Dio` ou `http`) pour centraliser les headers d'authentification.
- **Dépôts (Repositories)** : Injecter une couche d'abstraction entre la base de données locale et l'interface utilisateur.
- **Configuration** : Injecter des clés d'API ou des paramètres d'environnement.

5. Bonnes pratiques

- **Utilisez** `listen: false` : Pour les services qui ne changent jamais (comme un `ApiService`), utilisez toujours `listen: false` lors de la récupération. Cela évite des reconstructions inutiles du widget.
- **Interfaces (Abstract classes)** : Injectez des interfaces plutôt que des classes concrètes. Cela permet de basculer entre une implémentation `ProductionApiService` et `MockApiService` sans modifier le code des widgets.
- **MultiProvider** : Pour injecter plusieurs services, utilisez `MultiProvider` afin d'éviter l'imbrication profonde de widgets.

```
MultiProvider(  
  providers: [  
    Provider<ApiService>(create: (_) => ApiService()),  
    Provider<AuthRepository>(create: (_) => AuthRepository()),  
  ],  
  child: MyApp(),  
);
```

6. Flux de dépendances



Sources

- [Pub.dev - Provider package documentation](#)
- [Flutter Documentation - Dependency Injection](#)